

Fix and extend a broken webhook ingestion service

A small Go service that receives call-completion webhooks is running in production and misbehaving. Your job is to work out why, fix it, and make it safe to retry.

ROLE	TIME BOX	LANGUAGE	DUE
Backend Engineering Intern	4 hours	Go	3 days from receipt

1 What this is

You will receive a zip containing `webhook-ingest`, a Go service that accepts call-completion events from our telephony provider, stores them, and maintains per-account call statistics. It ships with Docker Compose, Postgres, Redis, migrations, and a test suite.

Everything builds and runs on first try. The tests pass. The service is nevertheless broken in ways the tests do not catch — which is the point of the exercise.

We are not looking for a rewrite. We are looking for someone who can read unfamiliar code, reason about what happens at runtime, and explain their thinking afterwards.

2 The incident

Last week our operations team filed this report:

OPS → ENGINEERING

Duplicate call records are showing up in the dashboard, and account call-counts are drifting higher than the actual number of calls. Calls are landing but their recordings never get marked processed — and there's nothing in the logs about it. On top of that, every time we deploy, whatever was in flight seems to just disappear.

Nobody has had time to look into it. That is your starting point — those symptoms, and the code.

WORTH KNOWING

The test suite in the repository does **not** cover everything that is broken. A green test run does not mean you are finished.

3 What we would like you to do

1 Find and fix the defects

Start from the symptoms above and work back to causes. Where you can, add a test that demonstrates the problem *before* you fix it — a test that fails first and passes after tells us far more than the fix alone.

2 Make ingestion idempotent

Our provider delivers **at least once**. It retries any non-2xx response, and it will occasionally redeliver an event even after a `200`. The `event_id` field is stable across redeliveries.

Ingesting the same event twice must not double-count anything. *How* you guarantee that is your design decision — Postgres and Redis are both running and connected. Pick an approach and be ready to defend it.

3 Write a short `SOLUTION.md`

About half a page, covering three things:

- What was broken, and why
- Why you chose your deduplication strategy over the alternatives you considered
- What you would change if this had to handle 10,000 webhooks/second

4 Getting started

You need Docker and Go 1.25 or newer. Nothing else.

```
unzip webhook-ingest.zip
cd webhook-ingest

docker compose up -d --build    # Postgres, Redis, and the service
curl localhost:8080/healthz    # -> ok
go test ./...                  # the visible test suite
```

`make reset` tears everything down, wipes the volumes, and starts fresh. Migrations are plain SQL in `migrations/`, applied by Postgres on first start of an empty volume — add new ones as `002_*.sql` and run `make reset`.

Already running Postgres or Redis locally?

The published host ports are overridable. Copy `.env.example` to `.env`, change `APP_PORT`, `POSTGRES_PORT` and `REDIS_PORT`, then point `DATABASE_URL` and `REDIS_ADDR` at your chosen ports so `go test ./...` finds them.

5 The API

POST /webhooks/calls

```
{
  "event_id":      "evt_01H8XK2M9P",
  "call_id":       "call_9f2ab31c",
  "account_id":    "acc_123",
  "status":        "completed",
  "duration_sec":  143,
  "recording_url": "https://recordings.example.com/9f2ab31c.wav",
  "occurred_at":   "2026-08-13T09:12:00Z"
}
```

`status` is one of `completed`, `failed`, `no_answer`.

ENDPOINT	PURPOSE
POST /webhooks/calls	Receive one call-completion event
GET /accounts/{id}/stats	Per-account totals from the in-memory aggregate. The durable copy of the same numbers lives in the <code>account_stats</code> table.
GET /healthz	Liveness check

Project layout

cmd/server/	entrypoint and wiring
internal/config/	environment configuration
internal/store/	Postgres repository
internal/stats/	in-memory per-account totals
internal/ingest/	webhook ingestion and processing
internal/httpapi/	routes and handlers
internal/redisclient/	Redis connection (connected; nothing uses it yet)
internal/testutil/	shared test setup
migrations/	schema

6 Ground rules

Time box: 4 hours

We would rather see two defects genuinely understood than four papered over. If you run out of time, say what you would have done next in `SOLUTION.md` — that counts.

AI tools are allowed

Use whatever you normally use. We are not testing whether you can work without them. We will spend 30 minutes walking through your code together afterwards, so make sure you can explain why every change you kept is correct.

Please don't change

- The entrypoint path `./cmd/server`
- The `BUILD_FLAGS` argument in the `Dockerfile`

Our tooling depends on both. Everything else is yours to restructure.

What we are not testing

Authentication, webhook signature verification, rate limiting, dashboards, and micro-optimisation are all out of scope. If you think one of them matters, mention it in `SOLUTION.md` rather than building it.

IF YOU GET STUCK ON SETUP

The infrastructure works out of the box. If you are fighting Docker for more than fifteen minutes, email hr@convin.ai instead of burning your time box on it. That will not count against you.

7 How we evaluate

We run an automated suite you have not seen against your submission, then read your code and your write-up. Roughly:

WHAT	WEIGHT
Defects genuinely fixed	60%
Tests you added — especially one that fails before your fix and passes after	15%
<code>SOLUTION.md</code> — clarity of reasoning, and how well you defend your design choice	15%
Commit history — incremental, with messages that say what and why	10%

Then a **30-minute conversation** about what you built. We will ask you to walk through a fix and explain why it is correct, and we will change a constraint on you live to see how you reason. This conversation carries real weight — a partial submission you can explain precisely beats a complete one you cannot.

8 Submitting

Push your work to a **public GitHub repository** and send us the link at hr@convin.ai — deadline **3 days from receipt**.

Commit as you go. We read the history, and a series of small commits with clear messages tells us more about how you work than one large final commit. Please include `SOLUTION.md` in the repository root.

Good luck — we are looking forward to reading it.